LeighNSteve Explorer Hang — Incident Writeup

Nov 5–6, 2026 · Host: LeighNSteve (Windows) · Prepared for Steven Powell · Prepared by Grok Bot

1. Executive summary

> **Bottom line:** Windows wasn't broken. The disk, CPU and memory were fine. File Explorer and every Open/Save dialog were stalling because **several cloud-sync clients (Google Drive, OneDrive, Dropbox, iCloud) had all loaded shell extensions into Explorer**: icon overlays, status handlers, context menus and copy hooks. Explorer sat waiting on them. Demoting and blocking those extensions (reversibly), then restarting Explorer, fixed it. **You didn't need to reinstall Windows, and the lasting fix didn't even need a reboot.**

It took about 6 hours of fighting a ~2 GB copy to USB and roughly 3 hours of joint debugging to get here. The good news is that the cause is understood, the changes can be undone, and the sync apps still sync. What changed is that they no longer get to hook into every folder view and file dialog.

2. Symptoms

- Explorer windows and **Save As / Open** dialogs sat on *"Working on it…"* for up to about a minute before showing their contents.
- The taskbar and Explorer froze intermittently, and the shell would go unresponsive while a folder was opening.
- **CPU stayed low while it was frozen.** That's the giveaway for a wait-blocked process (stuck on a lock, an RPC call or an out-of-process handler) as opposed to a compute-bound one. The machine wasn't busy. It was waiting.
- Copying ~2 GB to a USB drive turned into roughly 6 hours of misery. Each Explorer step (browse, drag, progress, confirm) paid the same stall.

3. Investigation

3.1 Ruling out the hardware and the OS

The first job was to show this wasn't a dying disk or a starved machine:

- **Disk:** healthy, with about 16 GB free and the disk queue near 0. Storage wasn't the bottleneck.
- **CPU / RAM:** healthy and low during the hangs, which fits the wait-blocked pattern above.
- **Raw directory listings were fast.** Enumerating the same folders from the command line came back in milliseconds. The filesystem answered right away. Only the *shell* (Explorer's view layer) was slow.

That pointed straight at the code Explorer loads to *decorate* a folder: overlays, property and status handlers, context menus and namespace extensions. Third-party software gets into the shell through exactly those hooks.

3.2 What was found in the shell

Finding	Why it matters
Two **googledrivefs** drivers present (earlier)	A duplicate or stale Google Drive filesystem driver next to the current one invites conflicts and slow G: enumeration.
17+ icon overlay handlers registered	Windows only loads about 15 overlay slots. Past that, load order is effectively a contest, and each loaded handler is queried for every icon drawn.
OneDrive **Status** / **FileSyncShell** handlers	Called on file views and dialogs to report sync state. A slow reply blocks the view.
Dropbox context menus and **copy hooks**	Copy hooks run on file copy/move operations, which fits the USB copy pain.
iCloud namespace extension	Another cloud provider wired into the navigation pane and every file dialog.
WMI intermittent hangs	A separate source of slowness. Anything querying WMI could stall.
xTend / Ativa USB utility **BEX64** crashes	Secondary. The vendor USB utility was crashing (buffer-overrun/DEP-type faults). It's noise around USB but not the main cause.

Working theory: each of these is tolerable alone. Stacked together on one machine, they meant every folder view and every `IFileDialog` (any app's Open/Save) had to wait on several sync clients answering “what's the status of this file?” If one was slow, the whole UI waited.

4. Remediation timeline

The changes were made in layers, from least to most invasive, so we could see what actually moved the needle:

Step	Action	Notes
1	**Google Drive** wiped and clean-reinstalled at **133.0.2.0** with a **single** googledrivefs driver	Removed the duplicate-driver condition. G: came back clean.
2	**Overlay demotions:** excess overlay handler keys renamed with a `zzz_DISABLED_*` prefix	Pushes them to the bottom of load order and past the ~15 slot limit. Reversible by renaming back.
3	**Shell Extensions\Blocked** entries for the OneDrive FileSync handlers	The documented Windows mechanism for keeping Explorer from loading a specific extension CLSID.
4	Explorer **Bags** (cached folder view state) cleared	Removes stale or corrupt per-folder view settings.
5	**Overnight:** HKCU-level (per-user) demotions applied without elevation	Covered everything reachable without admin rights.
6	**Morning:** `overnight-demote.ps1` run under UAC for the **HKLM** Dropbox / FileSyncEx registrations	The machine-wide entries needed admin rights, and the one UAC prompt was approved in the morning.
7	**Explorer restart**	This was enough for the changes to take effect. No OS reinstall, and the fix itself didn't depend on a reboot.

5. Outcome

- **Downloads and C:** open in about **0.8 s** after the fixes, and about **0.01 s** after the later reboot. A minute of “Working on it…” became effectively instant.
- **Google Drive (G:)** is working normally on the single clean driver.
- **The sync apps still work.** Google Drive, OneDrive and Dropbox keep syncing. Their overlays and context menus are turned off, so you lose the little sync-status badges and some right-click entries. That's a fair trade.
- **Heads-up:** OneDrive may ask you to sign in again after a reboot. That's expected and harmless.

6. Commentary

The real culprit is architecture, not a bug. Cloud sync clients hook into the Windows shell in-process. That means they load into Explorer *and* into every application that opens a standard Open/Save dialog (`IFileDialog`). Run three or four of them and every dialog in every app waits on all of them. Each vendor tests its client alone. Nobody tests Google Drive + OneDrive + Dropbox + iCloud on one box.

There were more overlays than Windows can load. With 17+ overlay handlers competing for about 15 slots, some were silently dropped and the rest were all asked about every icon. Overlay creep is one of the most common causes of a sluggish Explorer on machines that have built up sync tools over the years.

“Just reinstall Windows” would have been the expensive wrong answer. A reinstall would have cost hours of setup and reconfiguration. Then, as soon as Drive, OneDrive, Dropbox and iCloud were reinstalled, the same extensions would have come back and so would the hangs. The low-CPU, fast-raw-listing, slow-shell pattern said early on that this was a shell-extension problem and not an OS problem.

Reversible demotions beat nuking things. Renaming keys to `zzz_DISABLED_*` and using the Blocked list leaves the apps installed and working, and each change can be undone by putting one key back. The scripts in section 7 also keep backups.

Residual items (optional)

- **WMI hangs:** still worth an elevated look (repository consistency check / provider cleanup) if they keep showing up.
- **xTend / Ativa USB utility:** the BEX64 crashes suggest uninstalling or updating it with admin rights. It isn't needed for plain USB mass-storage copies.
- **Dropbox** may hang briefly when it closes. That's cosmetic and not a regression of this issue.
- **Software updates may re-register extensions.** If “Working on it…” comes back after a sync client updates, re-run the demotion script first.

7. Artifacts

Everything used or produced is in `C:\ProgramData\PowellServerOps\` on LeighNSteve:

Item	Purpose
`overnight-demote.ps1`	Demotion script: overlay renames, Blocked-list entries, Dropbox/FileSyncEx handling (HKCU and, when elevated, HKLM).
`morning-elevated.ps1`	Elevated follow-up pass for the machine-wide (HKLM) items.

Item	Purpose
Logs	A record of what each run changed.
Registry backups	Exports made before the changes, for rollback.
`LeighNSteve-Explorer-Incident-Writeup.pdf`	This document.

Rollback in one sentence: import the saved registry backups (or rename the `zzz_DISABLED_*` keys back and remove the Blocked entries), then restart Explorer.